Read Data - SELECT

We already have a database and a table with some data in it that looks more or less like this:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

Things are getting more exciting! Let's now see how to read data from the database! 🤩

Continue From Previous Code

Let's continue from the last code we used to create some data.

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

We are creating a **SQLModel** `Hero` class model and creating some records.

We will need the `Hero` model and the **engine**, but we will create a new session to query data in a new function.

Read Data with SQL

Before writing Python code let's do a quick review of how querying data with SQL looks like:

```sql
SELECT id, name, secret_name, age
FROM hero
```

It means, more or less:

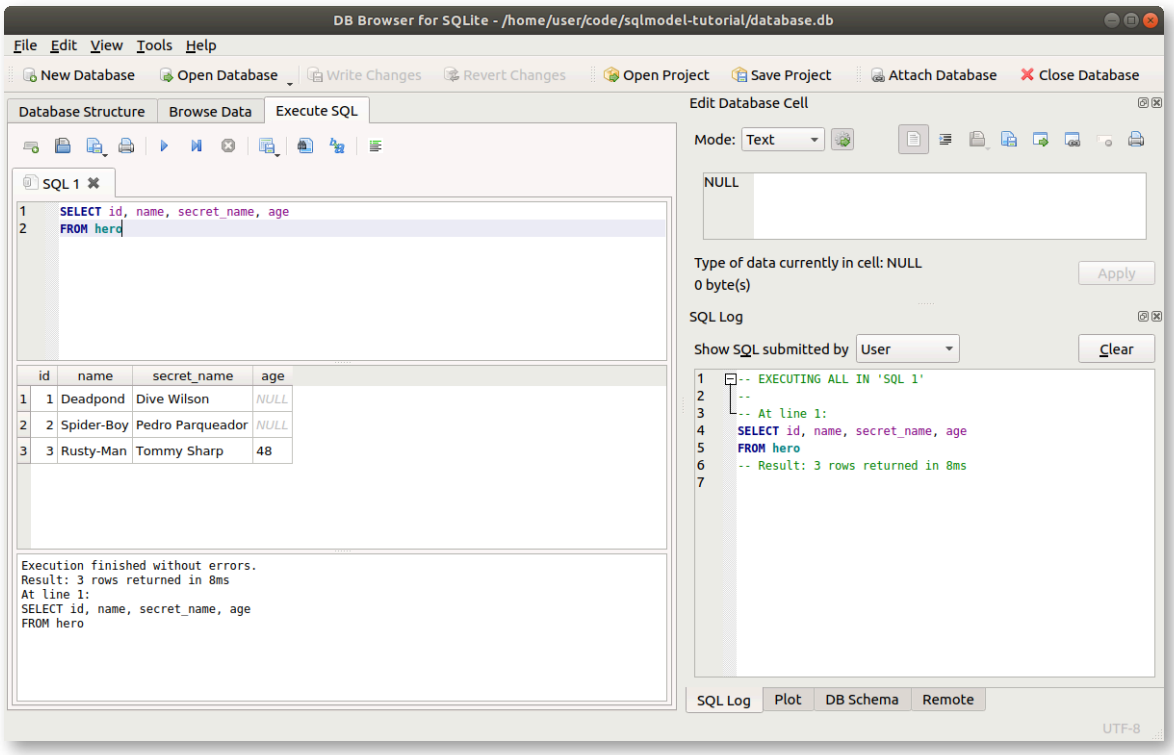
> Hey SQL database 👋, please go and `SELECT` some data for me.
>
> I'll first tell you the columns I want:
>
> - `id`
> - `name`

> - `secret_name`
> - `age`
>
> And I want you to get them `FROM` the table called `"hero"`.

Then the database will go and get the data and return it to you in a table like this:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

You can try that out in **DB Browser for SQLite**:



Warning

Here we are getting all the rows.

If you have thousands of rows, that could be expensive to compute for the database.

You would normally want to filter the rows to receive only the ones you want. But we'll learn about that later in the next chapter.

A SQL Shortcut

If we want to get all the columns like in this case above, in SQL there's a shortcut, instead of specifying each of the column names we could write a `*`:

```
SELECT *
FROM hero
```

That would end up in the same result. Although we won't use that for **SQLModel**.

`SELECT` Fewer Columns

We can also `SELECT` fewer columns, for example:

```
SELECT id, name
FROM hero
```

Here we are only selecting the `id` and `name` columns.

And it would result in a table like this:

id	name
1	Deadpond
2	Spider-Boy
3	Rusty-Man

And here is something interesting to notice. SQL databases store their data in tables. And they also always communicate their results in **tables**.

`SELECT` Variants

The SQL language allows several **variations** in several places.

One of those variations is that in `SELECT` statements you can use the names of the columns directly, or you can prefix them with the name of the table and a dot.

For example, the same SQL code above could be written as:

```
SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
```

This will be particularly important later when working with multiple tables at the same time that could have the same name for some columns.

For example `hero.id` and `team.id`, or `hero.name` and `team.name`.

Another variation is that most of the SQL keywords like `SELECT` can also be written in lowercase, like `select`.

Result Tables Don't Have to Exist

This is the interesting part. The tables returned by SQL databases **don't have to exist** in the database as independent tables. 🧙

For example, in our database, we only have one table that has all the columns, `id`, `name`, `secret_name`, `age`. And here we are getting a result table with fewer columns.

One of the main points of SQL is to be able to keep the data structured in different tables, without repeating data, etc, and then query the database in many ways and get many different tables as a result.

Read Data with SQLModel

Now let's do the same query to read all the heroes, but with **SQLModel**.

Create a Session

The first step is to create a **Session**, the same way we did when creating the rows.

We will start with that in a new function `select_heroes()`:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Create a `select` Statement

Next, pretty much the same way we wrote a SQL `SELECT` statement above, now we'll create a **SQLModel** `select` statement.

First we have to import `select` from `sqlmodel` at the top of the file:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

And then we will use it to create a `SELECT` statement in Python code:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select

# Code here omitted 👈

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

It's a very simple line of code that conveys a lot of information:

```python
statement = select(Hero)
```

This is equivalent to the first SQL `SELECT` statement above:

```sql
SELECT id, name, secret_name, age
FROM hero
```

We pass the class model `Hero` to the `select()` function. And that tells it that we want to select all the columns necessary for the `Hero` class.

And notice that in the `select()` function we don't explicitly specify the `FROM` part. It is already obvious to **SQLModel** (actually to SQLAlchemy) that we want to select `FROM` the table `hero`, because that's the one associated with the `Hero` class model.

> **Tip**
>
> The value of the `statement` returned by `select()` is a special object that allows us to do other things.
>
> I'll tell you about that in the next chapters.

Execute the Statement

Now that we have the `select` statement, we can execute it with the **session**:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

This will tell the **session** to go ahead and use the **engine** to execute that `SELECT` statement in the database and bring the results back.

Because we created the **engine** with `echo=True`, it will show the SQL it executes in the output.

This `session.exec(statement)` will generate this output:

```
INFO Engine BEGIN (implicit)
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
INFO Engine [no key 0.00032s] ()
```

The database returns the table with all the data, just like above when we wrote SQL directly:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

Iterate Through the Results

The `results` object is an iterable that can be used to go through each one of the rows.

Now we can put it in a `for` loop and print each one of the heroes:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

This will print the output:

```
id=1 name='Deadpond' age=None secret_name='Dive Wilson'
id=2 name='Spider-Boy' age=None secret_name='Pedro Parqueador'
id=3 name='Rusty-Man' age=48 secret_name='Tommy Sharp'
```

Add `select_heroes()` to `main()`

Now include a call to `select_heroes()` in the `main()` function so that it is executed when we run the program from the command line:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Review The Code

Great, you're now being able to read the data from the database! 🎉

Let's review the code up to this point:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select  ❶


class Hero(SQLModel, table=True):  ❷
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)  ❸


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)  ❹


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")  ❺
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:  ❻
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:  ❼
        statement = select(Hero)  ❽
        results = session.exec(statement)  ❾
        for hero in results:  ❿
            print(hero)  ⓫
    ⓬


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()  ⓭


if __name__ == "__main__":
    main()
```

❶ Import from `sqlmodel` everything we will use, including the new `select()` function.

❷ Create the `Hero` class model, representing the `hero` table.

❸ Create the **engine**, we should use a single one shared by all the application code, and that's what we are doing here.

❹ Create all the tables for the models registered in `SQLModel.metadata`.

This also creates the database if it doesn't exist already.

5. Create each one of the `Hero` objects.

   You might not have this in your version if you had already created the data in the database.

6. Create a new **session** and use it to `add` the heroes to the database, and then `commit` the changes.

7. Create a new **session** to query data.

> **Tip**
>
> Notice that this is a new **session** independent from the one in the other function above.
>
> But it still uses the same **engine**. We still have one engine for the whole application.

8. Use the `select()` function to create a statement selecting all the `Hero` objects.

   This selects all the rows in the `hero` table.

9. Use `session.exec(statement)` to make the **session** use the **engine** to execute the internal SQL statement.

   This will go to the database, execute that SQL, and get the results back.

   It returns a special iterable object that we put in the variable `results`.

   This generates the output:

```
INFO Engine BEGIN (implicit)
INFO Engine SELECT hero.id, hero.name, hero.secret_name, hero.age
FROM hero
INFO Engine [no key 0.00032s] ()
```

10. Iterate for each `Hero` object in the `results`.

11. Print each `hero`.

    The 3 iterations in the `for` loop will generate this output:

```
id=1 name='Deadpond' age=None secret_name='Dive Wilson'
id=2 name='Spider-Boy' age=None secret_name='Pedro Parqueador'
id=3 name='Rusty-Man' age=48 secret_name='Tommy Sharp'
```

12. At this point, after the `with` block, the **session** is closed.

    This generates the output:

```
INFO Engine ROLLBACK
```

13. Add this function `select_heroes()` to the `main()` function so that it is called when we run this program from the command line.

> **Tip**
>
> Check out the number bubbles to see what is done by each line of code.

Here it starts to become more evident why we should have a single **engine** for the whole application, but different **sessions** for each group of operations.

This new session we created uses the *same* **engine**, but it's a new and independent **session**.

The code above creating the models could, for example, live in a function handling web API requests and creating models.

And the second section reading data from the database could be in another function for other requests.

So, both sections could be in **different places** and would need their own sessions.

> **Info**
>
> To be fair, in this example all that code could actually share the same **session**, there's actually no need to have two here.
>
> But it allows me to show you how they could be separated and to reinforce the idea that you should have **one engine** per application, and **multiple sessions**, one per each group of operations.

Get a List of `Hero` Objects

Up to now we are using the `results` to iterate over them.

But for different reasons you might want to have the full **list of `Hero`** objects right away instead of just an *iterable*. For example, if you want to return them in a web API.

The special `results` object also has a method `results.all()` that returns a list with all the objects:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero)
        results = session.exec(statement)
        heroes = results.all()
        print(heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

With this now we have all the heroes in a list in the `heroes` variable.

After printing it, we would see something like:

```
[
    Hero(id=1, name='Deadpond', age=None, secret_name='Dive Wilson'),
    Hero(id=2, name='Spider-Boy', age=None, secret_name='Pedro Parqueador'),
    Hero(id=3, name='Rusty-Man', age=48, secret_name='Tommy Sharp')
]
```

Info

It would actually look more compact, I'm formatting it a bit for you to see that it is actually a list with all the data.

Compact Version

I have been creating several variables to be able to explain to you what each thing is doing.

But knowing what is each object and what it is all doing, we can simplify it a bit and put it in a more compact form:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        print(heroes)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        heroes = session.exec(select(Hero)).all()
        print(heroes)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Here we are putting it all on a single line, you will probably put the select statements in a single line like this more often.

SQLModel or SQLAlchemy - Technical Details

SQLModel is actually, more or less, just **SQLAlchemy** and **Pydantic** underneath, combined together.

It uses and returns the same types of objects and is compatible with both libraries.

Nevertheless, **SQLModel** defines a few of its own internal parts to improve the developer experience.

In this chapter we are touching some of them.

SQLModel's `select`

When importing from `sqlmodel` the `select()` function, you are using **SQLModel**'s version of `select`.

SQLAchemy also has its own `select`, and SQLModel's `select` uses SQLAlchemy's `select` internally.

But SQLModel's version does a lot of **tricks** with type annotations to make sure you get the best **editor support** possible, no matter if you use **VS Code**, **PyCharm**, or something else. ✨

> **Info**
>
> There was a lot of work and research, with different versions of the internal code, to improve this as much as possible. 🤓

SQLModel's `session.exec`

📢 This is one to pay special attention to.

SQLAlchemy's own `Session` has a method `session.execute()`. It doesn't have a `session.exec()` method.

If you see SQLAlchemy tutorials, they will always use `session.execute()`.

SQLModel's own `Session` inherits directly from SQLAlchemy's `Session`, and adds this additional method `session.exec()`. Underneath, it uses the same `session.execute()`.

But `session.exec()` does several **tricks** combined with the tricks in `session()` to give you the **best editor support**, with **autocompletion** and **inline errors** everywhere, even after getting data from a select. ✨

For example, in SQLAlchemy you would need to add a `.scalars()` here:

```
heroes = session.execute(select(Hero)).scalars().all()
```

But you would have to remove it when selecting multiple things (we'll see that later).

SQLModel's `session.exec()` takes care of that for you, so you don't have to add the `.scalars()`.

This is something that SQLAlchemy currently can't provide, because the regular `session.execute()` supports several other use cases, including legacy ones, so it can't have all the internal type annotations and tricks to support this.

On top of that, **SQLModel**'s `session.exec()` also does some tricks to reduce the amount of code you have to write and to make it as intuitive as possible.

But SQLModel's `Session` still has access to `session.execute()` too.

> **Tip**
>
> Your editor will give you autocompletion for both `session.exec()` and `session.execute()`.
>
> 👀 Remember to **always use** **`session.exec()`** to get the best editor support and developer experience.

Caveats of **SQLModel** Flavor

SQLModel is designed to have the best **developer experience** in a narrow set of **very common use cases**. ✨

You can still combine it with SQLAlchemy directly and use **all the features** of SQLAlchemy when you need to, including lower level more "pure" SQL constructs, exotic patterns, and even legacy ones. 🤓

But **SQLModel**'s design (e.g. type annotations) assume you are using it in the ways I explain here in the documentation.

Thanks to this, you will get as much **autocompletion** and **inline errors** as possible. 🚀

But this also means that if you use SQLModel with some more **exotic patterns** from SQLAlchemy, your editor might tell you that *there's an error*, while in fact, the code would still work.

That's the trade-off. 🤷

But for the situations where you need those exotic patterns, you can always use SQLAlchemy directly combined with SQLModel (using the same models, etc).